**VIETNAM NATIONAL UNIVERSITY**
**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**
**FACULTY OF COMPUTER SCIENCE AND ENGINEERING**




**Parallel Computing**

---

**ASSIGNMENT 1**

---

**Instructor(s):** Le Thanh Sach

**Group Members:**

Tran Ngoc Dat - 2570388

Nguyen Dinh Quang Khai - 2570231

HO CHI MINH, October, 2026

**Table Contents**

**Table Contents....2**
**1. Dataset....3**
1.1 CIFAR-100 Dataset Overview....3
**2. Assignment 1....5**
2.1 Data split....5
2.2 Data augmentation....5
2.3 Transformation Plan....6
2.3.1 Overview of ViT....6
2.3.2 ViT configuration....6
2.3.3 Training Plan: From-Scratch, Pretrained, and Freeze Strategies....7
2.3.3.1 ViT From-Scratch....7
2.3.3.2 Pretrained ViT - Freeze Entire Backbone....7
2.3.3.3 Pretrained ViT - Partial Freeze....7
2.3.3.4 Pretrained ViT - Full Fine-Tuning....8
2.4 ResNet Architecture....9
2.4.1 Introduction....9
2.4.2 ResNet From-Scratch....9
2.4.2.1 Basic Residual Block Implementation....9
2.4.2.1.1 Basic Residual Block Configuration....9
2.4.2.1.1 Forward Implementation....11
2.4.2.2 ResNet Implementation....11
2.4.2.2 Discussion And Experimental Analysis....12
2.4.2.3 Analysis....12
2.4.3 ResNet Pretrained....13
2.4.3.1 ResNet Pretrained - Freeze All Of Backbone....13
2.4.3.1 ResNet Pretrained - Freeze Partial Backbone....13
2.4.3.1 ResNet Pretrained - Full Finetune....13
**3. References....13**

# 1. Dataset

## 1.1 CIFAR-100 Dataset Overview

CIFAR-100 is a widely used benchmark dataset for image classification. It contains 60,000 color images of size 32 × 32 pixels, divided into 100 object categories. Each category contains 600 images, resulting in a balanced class distribution.

License: Open Source / Academic License (Publicly available for research and education purposes).

The dataset is split into:

- Training set: 50,000 images (500 images per class)
- Test set: 10,000 images (100 images per class)

The images cover a diverse range of objects, including animals, vehicles, household items, plants, and natural scenes, making CIFAR-100 significantly more challenging than CIFAR-10 due to its larger number of categories and fewer samples per class.

Reason for Choosing CIFAR-100:
Constraint Compliance: CIFAR-100 is a publicly available color image dataset, distinct from simpler datasets like MNIST or CIFAR-10 (which were used in assignment E1).
Complexity and Class Count: It features 100 fine-grained classes, offering a challenging benchmark for Vision Transformers (ViT) that typically demand rich data variety to learn spatial representations from scratch.
Data Scale: Contains 60,000 32x32 color images. This dataset is large enough to train deep learning architectures while remaining computationally feasible for standard GPU resources.

| **Property** | **Value** |
|---|---|
| Dataset | CIFAR-100 |
| Total Images | 60,000 |
| Training Images | 50,000 |
| Test Images | 10,000 |
| Number of Classes | 100 |
| Images per Class | 600 |
| Image Resolution | 32 × 32 RGB |
| Fine Labels | 100 |
| Coarse Labels | 20 |

| Superclass | Classes |
|---|---|
| aquatic mammals | beaver, dolphin, otter, seal, whale |
| fish | aquarium fish, flatfish, ray, shark, trout |
| flowers | orchids, poppies, roses, sunflowers, tulips |
| food containers | bottles, bowls, cans, cups, plates |
| fruit and vegetables | apples, mushrooms, oranges, pears, sweet peppers |
| household electrical devices | clock, computer keyboard, lamp, telephone, television |
| household furniture | bed, chair, couch, table, wardrobe |
| insects | bee, beetle, butterfly, caterpillar, cockroach |
| large carnivores | bear, leopard, lion, tiger, wolf |
| large man-made outdoor things | bridge, castle, house, road, skyscraper |
| large natural outdoor scenes | cloud, forest, mountain, plain, sea |
| large omnivores and herbivores | camel, cattle, chimpanzee, elephant, kangaroo |
| medium-sized mammals | fox, porcupine, possum, raccoon, skunk |
| non-insect invertebrates | crab, lobster, snail, spider, worm |
| people | baby, boy, girl, man, woman |
| reptiles | crocodile, dinosaur, lizard, snake, turtle |
| small mammals | hamster, mouse, rabbit, shrew, squirrel |
| trees | maple, oak, palm, pine, willow |
| vehicles 1 | bicycle, bus, motorcycle, pickup truck, train |
| vehicles 2 | lawn-mower, rocket, streetcar, tank, tractor |

# 2. Assignment 1

## 2.1 Data split

The CIFAR-100 dataset provides an official partition consisting of 50,000 training images and 10,000 test images. Following common practice in image classification research, the original test set is preserved exclusively for final model evaluation to ensure an unbiased assessment of generalization performance.
To facilitate model selection and hyperparameter tuning, the training set is further divided into a training subset (80%) and a validation subset (20%) using stratified sampling. This results in approximately 40,000 training images and 10,000 validation images, while maintaining a similar class distribution across all subsets.

| Subset | Number of Images | Purpose |
|---|---|---|
| Training | 40,000 | Model training |
| Validation | 10,000 | Hyperparameter tuning and model selection |
| Testing | 10,000 | Final evaluation |

This split provides a sufficiently large validation set for reliable model selection and performance monitoring while retaining enough training samples to learn meaningful representations. The validation set is used to compare different architectures, tune training hyperparameters, and identify the optimal checkpoint. The test set remains unseen throughout the training and tuning process, thereby preventing data leakage and ensuring a fair evaluation of model generalization capability.

Given the relatively challenging nature of CIFAR-100, where each class contains only 500 training samples in the original dataset, maintaining a dedicated validation subset is essential for detecting overfitting and evaluating the robustness of both CNN-based and transformer-based models.

## 2.2 Data augmentation

The image processing pipeline consists of two distinct workflows for training and evaluation:

- Train Split: Raw Image -> Random Crop -> Normalization -> Model Input
- Test/Val Split: Raw Image -> Normalization -> Model Input

Using PyTorch's torchvision.transforms, the data augmentation pipeline for the training set consists of the following transformations:
Random Crop (transforms.RandomCrop)

- Parameters: size = 32, padding = 4, padding_mode = 'reflect'

- Mechanism: Expands the input 32x32 image to 40x40 via reflection padding, followed by a random 32x32 crop.
- Purpose: Introduces spatial translation invariance and prevents positional memorization.

Normalization (transforms.Normalize)

- Parameters:
  - Channel Mean: [0.5071, 0.4867, 0.4408]
  - Channel Std: [0.2675, 0.2565, 0.2761]
- Formula: X_norm = (X - mean) / std
- Purpose: Centers channel-wise distributions around zero mean and unit variance to stabilize backpropagation and prevent activation saturation in ViT block layers.

## 2.3 Transformation Plan

### 2.3.1 Overview of ViT

The central idea of ViT is that image understanding can be learned through global relationships between image regions. While CNNs focus primarily on local patterns through convolution operations, ViT can directly model long-range dependencies between distant parts of an image. This property makes Transformer-based models particularly attractive for complex image recognition problems containing many object categories and diverse visual features.

For this project, the selected Transformer architecture is **Vision Transformer (ViT)**, with a patch size suitable for CIFAR-100 images. The dataset contains 100 object classes and 60,000 color images, making it sufficiently challenging to highlight the differences between convolution-based and attention-based learning approaches.

### 2.3.2 ViT configuration

Transformer Architecture: Vision Transformer (ViT)

| Component | Configuration |
|---|---|
| Model | ViT |
| Input Resolution | 32 × 32 RGB |
| Patch Size | 4 × 4 |
| Number of Classes | 100 |
| Transformer Blocks | Standard Encoder Blocks |

| | |
|---|---|
| Attention Mechanism | Multi-Head Self-Attention (MSA) |
| Classification Head | Fully Connected Layer |
| Framework | PyTorch |

### 2.3.3 Training Plan: From-Scratch, Pretrained, and Freeze Strategies

#### 2.3.3.1 ViT From-Scratch

In the first configuration, all model weights are randomly initialized and trained entirely on CIFAR-100.
Purpose:

- Establish a baseline Transformer performance.
- Evaluate whether the dataset provides enough information for ViT to learn meaningful features without prior knowledge.
- Compare pure learning capability against ResNet trained from scratch.

Expected Observation: Because Transformers generally have weaker built-in inductive biases than CNNs, ViT may require more training data and longer training time to achieve competitive accuracy.

#### 2.3.3.2 Pretrained ViT - Freeze Entire Backbone

In this configuration, ImageNet-pretrained weights are used. All Transformer encoder layers remain frozen, and only the final classification head is trained.
Purpose:

- Measure the quality of learned ImageNet features.
- Minimize training time and computational cost.
- Evaluate the effectiveness of transfer learning when only a small number of parameters are updated.

Potential limitation: The frozen backbone may not adapt well to CIFAR-100 specific features, which can limit final accuracy.

#### 2.3.3.3 Pretrained ViT - Partial Freeze

For the partial freeze strategy, early Transformer layers remain frozen while later layers and the classification head are trainable.
Purpose

- Retain generic visual features from pretraining.
- Allow higher-level representations to adapt to CIFAR-100.
- Balance computational efficiency and model flexibility.

Expected observation: This strategy often provides better accuracy than full freezing while requiring significantly fewer trainable parameters than full fine-tuning.

#### 2.3.3.4 Pretrained ViT - Full Fine-Tuning

All pretrained parameters are trainable and updated during training.
Purpose

- Maximize adaptation to the target dataset.
- Fully exploit the benefits of transfer learning.

Potential limitation:

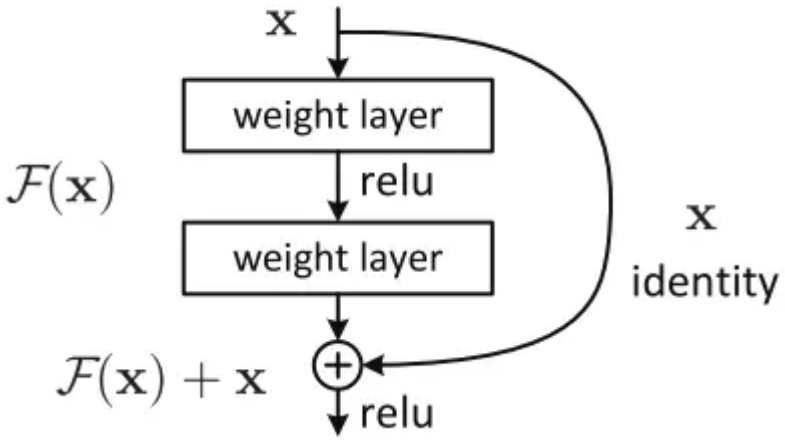
- Requires longer training time.
- Higher GPU memory usage.
- Increased risk of overfitting if training is not properly regularized.

# 2.4 ResNet Architecture

### 2.4.1 Introduction

The ResNet stands for Residual Network. This is a Neural Network Architecture designed to make training deep networks practical.
The core idea is Residual Block: **H(x) = x + F(x)**
**Where:**

- x: Input to block
- H(x): Desired mapping
- F(x): Residual function to be learned




The Residual Block

A **Skip Connection** is a direct path that bypasses one or more layers and feeds the input to a later point in the network.

In a traditional network, data flows through each layer in sequence. Every layer transforms the input and passes the result to the next one. Skip connections take the original input and add it directly to the output of a layer further down the block.

ResNet architecture is all about one thing: skip connections. That one idea solved two problems that had been holding deep networks back - vanishing gradients and the degradation problem - and made it practical to train networks at a depth that wasn't possible before.

### 2.4.2 ResNet From-Scratch

#### 2.4.2.1 Basic Residual Block Implementation

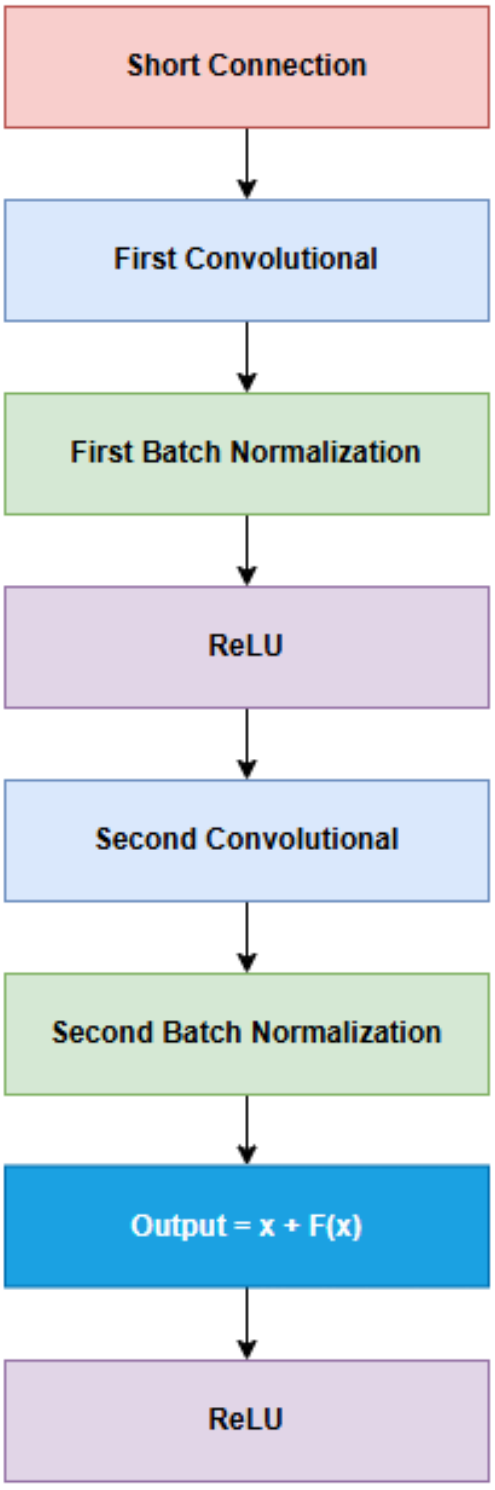
##### 2.4.2.1.1 Basic Residual Block Configuration

- The function of Basic Residual Block is defined as **BasicBlock(nn.Module)**

| Layer | Input | Output |
|---|---|---|
| **First Convolutional:** | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |

| | | |
|---|---|---|
| Conv2d(in_channels = <in_channels>, out_channels = <out_channels>, stride = <stride>, kernel_size = 3) | | |
| **First Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Second Convolutional:**<br>Conv2d(in_channels = <out_channels>, out_channels = <out_channels>, stride = 1, kernel_size = 3) | - tensor([N,<out_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) ) |
| **Second Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Short Connection:**<br>if(<sride>!= 1 OR <in_channels>!= <out_channels>)<br>⇒ Conv2d(in_channels = <in_channels>, out_channels = <out_channels>, stride = <stride>, kernel_size = 1)<br>⇒ BatchNorm2d(out_channels=<out_channels>)<br>else<br>⇒ Identity() | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |

#### 2.4.2.1.1 Forward Implementation




The Forward Implementation

### 2.4.2.2 ResNet Implementation

| Layer | Input | Output |
|---|---|---|

| | | |
|---|---|---|
| **First Convolutional:**<br>Conv2d(in_channels = 3, out_channels = 64, stride = <stride>, kernel_size = 3) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **First Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Second Convolutional:**<br>Conv2d(in_channels = <out_channels>, out_channels = <out_channels>, stride = 1, kernel_size = 3) | - tensor([N,<out_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) ) |
| **Second Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Short Connection:**<br>if(<sride>!= 1 OR <in_channels>!= <out_channels>)<br>⇒ Conv2d(in_channels = <in_channels>, out_channels = <out_channels>, stride = <stride>, kernel_size = 1)<br>⇒ BatchNorm2d(out_channels=<out_channels>)<br>else<br>⇒ Identity() | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |

#### 2.4.2.2 Discussion And Experimental Analysis

#### 2.4.2.3 Analysis

The table of training progress with the different **Learning Rate** and **Batch Size**
The number of **epoch** is **10**

| Learning Rate | Batch Size | Train Loss | Test Loss | Train Accuracy | Test Accuracy |
|---|---|---|---|---|---|
| 0.001 | 128 | | | | |
| | | | | | |
| | | | | | |

The Confusion Matrix of model with **Learning Rate** = , and **Batch Size** =

#### 2.4.3 ResNet Pretrained

##### 2.4.3.1 ResNet Pretrained - Freeze All Of Backbone

##### 2.4.3.1 ResNet Pretrained - Freeze Partial Backbone

##### 2.4.3.1 ResNet Pretrained - Full Finetune

# 3. References

Asad, Muhammad. “Introduction to Softmax Classifier in PyTorch - MachineLearningMastery.com.” *Machine Learning Mastery*, https://machinelearningmastery.com/introduction-to-softmax-classifier-in-pytorch/. Accessed 15 October 2026.